

RADEETA Management System

Application web de gestion des operations terrain, de la facturation et du pilotage de service pour SRM Taza/Region.

Date de génération 03/05/2026 18:59	Sources analysées frontend/ et backend/
Stack observée React + Vite / Laravel + Sanctum / MySQL	Type de document Rapport technique de présentation orienté métier

Analyse réalisée uniquement à partir du code source disponible dans le dépôt. Le rapport décrit l'architecture, les flux, les composants et la logique métier réellement visibles dans les fichiers.

Sommaire

- 1. Présentation du projet
- 2. Technologies utilisées
- 3. Architecture
- 4. Structure des dossiers
- 5. Fonctionnalités principales
- 6. Partie Frontend
- 7. Partie Backend
- 8. API (endpoints)
- 9. Base de données
- 10. Authentification et rôles
- 11. Workflow de l'application
- 12. Exemples de code importants
- 13. Conclusion

1. Présentation du projet

Ce document a été préparé à partir de l’analyse du code source du projet `RADEETA-Management-System` , sans modifier le code applicatif. Le produit exposé au travers du frontend porte le nom `SRM Taza/Region` et se présente comme une plateforme de gestion des opérations d’un service d’eau et d’assainissement.

Objectif du projet

- Centraliser dans une seule application la gestion des abonnés, des compteurs, des secteurs, des pannes, des réparations, des relevés, des factures et des paiements.
- Donner à chaque profil métier un espace de travail adapté : administration, supervision, terrain et consultation.
- Fournir un cycle complet allant de l’incident terrain jusqu’à la facturation et au reporting.

Problème résolu

- Évite la dispersion des informations entre plusieurs fichiers, outils ou traitements manuels.
- Sécurise l’accès aux données grâce à un contrôle d’accès par rôle à la fois côté frontend et côté backend.
- Automatise des règles métier sensibles : calcul de consommation, génération de facture, suivi du solde restant et clôture des pannes après réparation.
- Améliore le pilotage grâce aux dashboards, aux cartes GIS et aux exports PDF / Excel.

2. Technologies utilisées

Le projet est construit comme une application web full stack séparée en deux sous-projets : un frontend React/Vite et un backend Laravel. La base est relationnelle et la sécurité repose sur des tokens d’API.

Couche	Technologies	Rôle dans le projet
Frontend	React 19, Vite 8, Tailwind CSS, React Router 7, Axios, Recharts, Leaflet, Lucide React	Interface web SPA, navigation, cartes, tableaux, graphiques et consommation de l’API
Backend	Laravel 11 sur PHP 8.2	API REST, validation, contrôle d’accès, logique métier, export de documents
Base de données	MySQL via Eloquent ORM, migrations, factories et seeders	Stockage relationnel des données métier et du système

Couche	Technologies	Rôle dans le projet
Authentification	Laravel Sanctum + middleware personnalisé role + ProtectedRoute côté frontend	Connexion par token, sécurité des routes et contrôle des rôles
Documents	barryvdh/laravel-dompdf, maatwebsite/excel	Génération de PDF et export Excel
Qualité / outils	PHPUnit, ESLint, Vite, seeders de démonstration	Tests d'API, qualité du code et environnement de développement

La configuration locale observée dans le code pointe le frontend vers `http://127.0.0.1:8000/api` et utilise `MySQL` comme base de données principale.

3. Architecture

Structure générale du système



Communication entre les composants

1. L'utilisateur se connecte depuis le frontend React via la page `Login.jsx`.
2. Le frontend envoie les identifiants à `POST /api/login`.
3. Laravel vérifie le compte, crée un token Sanctum et renvoie le rôle ainsi que le profil utilisateur.
4. Le frontend stocke le token et le renvoie automatiquement dans l'en-tête `Authorization: Bearer ...` pour chaque appel ultérieur.
5. Les contrôleurs Laravel exécutent la logique métier, appellent si besoin les services et renvoient des ressources JSON structurées.
6. Les fichiers PDF et Excel sont générés côté backend puis téléchargés côté frontend sous forme de blob.

Les notifications ne passent pas par WebSocket dans cette version : la cloche du frontend effectue un polling régulier toutes les 30 secondes pour rafraîchir les données.

4. Structure des dossiers

```
RADEETA-Management-System/
|-- frontend/
|   |-- src/
|       |-- api/
|       |-- components/
|       |-- context/
|       |-- dashboards/
|       |-- hooks/
|       |-- layouts/
|       |-- pages/
|       |-- routes/
|       |-- `-- utils/
|   |-- `-- package.json
|
|-- `-- backend/
|   |-- app/
|       |-- Enums/
|       |-- Http/
|       |   |-- Controllers/
|       |   |-- Middleware/
|       |   |-- Requests/
|       |   |-- `-- Resources/
|       |-- Models/
|       |-- Notifications/
|       |-- Services/
|       |-- `-- Support/
|   |-- database/
|       |-- factories/
|       |-- migrations/
|       |-- `-- seeders/
|   |-- resources/views/
|   |-- routes/api.php
|   |-- `-- tests/Feature/
```

Fichier / dossier	Explication
frontend/src/App.jsx	Routeur principal : définit les écrans par rôle et le flux global de navigation.
frontend/src/context/AuthContext.jsx	Gestion de session, stockage local du token et redirection après connexion.

Fichier / dossier	Explication
<code>frontend/src/api/axios.js</code>	Client HTTP centralisé avec injection automatique du token Bearer.
<code>frontend/src/dashboards/DashboardShared.jsx</code>	Chargement mutualisé des données de dashboard et widgets communs.
<code>backend/routes/api.php</code>	Déclaration de toutes les routes REST et des règles d'accès par rôle.
<code>backend/app/Http/Controllers/*</code>	Contrôleurs des ressources métier, de l'authentification, des rapports et des notifications.
<code>backend/app/Services/BillingService.php</code>	Calcul des montants, des tranches, des taxes et création des factures.
<code>backend/app/Support/OperatorAccess.php</code>	Filtrage métier pour limiter les opérateurs à leurs données assignées.
<code>backend/database/migrations/*</code>	Historique de création et d'évolution du schéma relationnel.
<code>backend/database/seeder/DatabaseSeeder.php</code>	Jeu de données réaliste pour démonstration et tests fonctionnels.
<code>backend/resources/views/invoices/*</code> et <code>backend/resources/views/reports/*</code>	Templates Blade utilisés pour les PDF de factures et de rapports mensuels.

Le dépôt montre aussi des traces d'évolution progressive du projet, par exemple

`frontend/src/pages/Dashboard.jsx`, `frontend/src/pages/Notifications.jsx` ou `backend/resources/js/*`, qui ne sont pas au cœur du routage React actuel mais témoignent d'itérations précédentes.

5. Fonctionnalités principales

Le code source met en évidence un périmètre métier large couvrant le terrain, l'administration et la facturation.

Fonctionnalité	Explication
----------------	-------------

Authentification sécurisée	Connexion avec <code>identifiant</code> et <code>password</code> , génération d'un token Sanctum, déconnexion et expiration propre côté frontend.
Gestion des utilisateurs	Création et mise à jour des comptes avec rôles <code>super_admin</code> , <code>admin</code> , <code>manager</code> , <code>operator</code> , <code>viewer</code> , <code>developer</code> .
Gestion des clients	CRUD sur les abonnés avec police, identité, CIN, téléphone, adresse et statut d'abonnement.
Gestion des compteurs	Association des compteurs aux clients et aux secteurs, avec calibre, marque et index de lecture.
Gestion des secteurs	Découpage géographique par zone, tournée et coordonnées GPS pour l'affichage cartographique.
Suivi des pannes	Déclaration d'une panne, affectation à un opérateur et suivi du statut ouvert/résolu.
Gestion des réparations	Création d'une intervention liée à une panne ; une réparation clôt automatiquement la panne concernée.
Relevés de compteurs	Saisie des anciens/nouveaux index et calcul automatique de la consommation.
Facturation	Génération d'une facture à partir d'un relevé non encore facturé, avec détail des lignes, taxes, échéance et PDF.
Paielements	Enregistrement des règlements, calcul du solde restant et mise à jour automatique du statut de facture.
Notifications	Notifications base de données visibles dans la cloche du frontend et marquage comme lues.
Reporting	Exports PDF et Excel pour les pannes, factures, paiements, clients et compteurs.

6. Partie Frontend

Le frontend est une application SPA construite avec React et Vite. Il n'implémente pas la sécurité à lui seul : il améliore l'expérience utilisateur en masquant certaines actions, mais l'autorité finale reste côté API Laravel.

Page / composant	Rôle
------------------	------

Login.jsx	Écran de connexion ; appelle <code>AuthContext.login</code> puis redirige vers le dashboard du rôle connecté.
Layout.jsx , Sidebar.jsx , Topbar.jsx	Cadre principal de l'application avec navigation latérale, titre de page, rôle courant et cloche de notifications.
ProtectedRoute.jsx et RoleRedirect.jsx	Protection des routes et redirection automatique selon le rôle.
rbac.js	Règles frontend pour afficher ou masquer les actions CRUD et choisir les chemins de dashboard.
AdminDashboard.jsx , ManagerDashboard.jsx , OperatorDashboard.jsx , ViewerDashboard.jsx	Dashboards adaptés aux responsabilités de chaque type d'utilisateur.
Clients.jsx , Compteurs.jsx , Secteurs.jsx	Pages de gestion des référentiels métier via <code>DataTable</code> et <code>EntityFormModal</code> .
Pannes.jsx , Reparations.jsx , Relevés.jsx , MyTasks.jsx	Écrans opérationnels pour le terrain, l'affectation, les interventions et le travail quotidien des opérateurs.
Factures.jsx , Paiements.jsx , TariffSettings.jsx , Reports.jsx	Écrans de facturation, d'encaissement, de configuration tarifaire et de reporting.
DataTable.jsx	Composant générique de tableaux avec recherche, tri, pagination et actions conditionnées par le rôle.
PanneMap.jsx	Carte Leaflet des secteurs/pannes à partir des coordonnées GPS.
InvoiceDetailModal.jsx	Vue détaillée d'une facture avec lignes d'eau, assainissement, taxes et actions PDF.
NotificationBell.jsx	Chargement périodique des notifications (polling toutes les 30 secondes).

Observations importantes sur le frontend

- Les données sont chargées principalement via le hook générique `useResource` , qui gère `loading` , `error` , `items` et `refresh` .

- Les actions CRUD passent par un petit nombre de composants réutilisables : `DataTable` , `EntityFormModal` , `ConfirmDialog` , `Badge` , `Button` .
- La carte `PanneMap` s'appuie sur `react-leaflet` et utilise les coordonnées GPS des secteurs pour afficher les incidents.
- Les téléchargements de PDF et d'Excel se font via `downloadFile` , qui manipule les blobs reçus depuis l'API.
- Le routeur principal utilisé en production est `frontend/src/App.jsx` , qui oriente vers des dashboards par rôle (`admin` , `manager` , `operator` , `viewer`).

7. Partie Backend

Le backend Laravel organise clairement les responsabilités : contrôleurs pour exposer l'API, services pour les règles métier transverses, modèles Eloquent pour la persistance, ressources pour la forme JSON et middleware pour la sécurité.

Composant backend	Responsabilité
<code>AuthController</code>	Connexion, émission du token Sanctum et déconnexion.
<code>UserController</code>	Lecture, création et mise à jour des utilisateurs ; récupération ciblée des opérateurs.
<code>ClientController</code> , <code>CompteurController</code> , <code>SecteurController</code>	CRUD sur les référentiels principaux.
<code>PanneController</code>	Création, affectation, filtrage des opérateurs, changement de statut et notifications d'affectation.
<code>ReparationController</code> + <code>ReparationRequest</code>	Validation métier, clôture automatique de panne, contrôle de date et unicité de la réparation active.
<code>ReleveController</code>	Saisie des relevés, contrôle des index, calcul de consommation et protection des relevés déjà facturés.
<code>FactureController</code>	Liste, création, aperçu PDF et téléchargement des factures.
<code>PaieementController</code>	Ajout des paiements, contrôle du dépassement du solde et rafraîchissement du statut de facture.
<code>DashboardController</code>	Indicateurs de pilotage, métriques par secteur et top opérateurs.

Composant backend	Responsabilité
NotificationController	Lecture et marquage des notifications, avec visibilité selon le rôle.
ReportController	Exports mensuels PDF/Excel pour le management.
TariffSettingController	Lecture et mise à jour de la configuration tarifaire JSON.
BillingService	Calcule les tranches d'eau/assainissement, la TVA, les frais fixes et la structure détaillée d'une facture.
NotificationService	Diffuse les événements métier importants aux bons profils.
OperatorAccess + EnsureUserHasRole	Double sécurité métier : filtrage des données opérateur et contrôle d'accès aux endpoints.

Logique métier notable

- Une réparation valide ferme automatiquement la panne correspondante ; si la réparation disparaît, la panne peut être réouverte.
- Un relevé déjà facturé ne peut plus être modifié.
- Le montant d'un paiement ne peut pas dépasser le solde restant de la facture.
- Les opérateurs ne voient pas tout le système : `OperatorAccess` filtre leurs données à partir des pannes et secteurs qui leur sont assignés.
- La configuration tarifaire est stockée en JSON et fusionnée avec des valeurs par défaut pour éviter les paramètres manquants.

8. API (endpoints)

L'API suit une logique REST enrichie de quelques routes métier (authentification, prévisualisation PDF, rapports et paramètres tarifaires). Les routes sont regroupées dans `backend/routes/api.php` et protégées par `auth:sanctum` puis par le middleware `role`.

Endpoint principal	Catégorie	Utilité
<code>POST /api/login</code>	Authentification	Valide les identifiants, retourne le token, le rôle et le profil utilisateur.
<code>POST /api/logout</code>	Authentification	Supprime le token actif.
<code>GET /api/dashboard/summary</code> et <code>GET /api/dashboard/stats</code>	Pilotage	Retourne les statistiques globales du système.
<code>GET /api/notifications</code>	Notifications	Liste les notifications visibles par l'utilisateur connecté.
<code>POST /api/notifications/read</code> et <code>PUT /api/notifications/mark-as-read</code>	Notifications	Marque une ou plusieurs notifications comme lues.
<code>GET /api/clients</code> , <code>GET /api/clients/{id}</code>	Clients	Consultation des clients.
<code>POST /api/clients</code> , <code>PUT /api/clients/{id}</code> , <code>DELETE /api/clients/{id}</code>	Clients	Création, modification et suppression logique des clients.
<code>GET /api/compteurs</code> , <code>GET /api/compteurs/{id}</code>	Compteurs	Consultation des compteurs et de leurs relations.
<code>POST /api/compteurs</code> , <code>PUT /api/compteurs/{id}</code> , <code>DELETE /api/compteurs/{id}</code>	Compteurs	Gestion des compteurs.
<code>GET /api/secteurs</code> , <code>GET /api/secteurs/{id}</code>	Secteurs	Consultation des secteurs et des coordonnées GPS.
<code>POST /api/secteurs</code> , <code>PUT /api/secteurs/{id}</code> , <code>DELETE /api/secteurs/{id}</code>	Secteurs	Gestion des secteurs et tournées.

Endpoint principal	Catégorie	Utilité
GET /api/pannes , GET /api/pannes/{id}	Pannes	Lecture des pannes ; les opérateurs ne voient que leurs pannes assignées.
POST /api/pannes , PUT /api/pannes/{id} , DELETE /api/pannes/{id}	Pannes	Création, affectation, changement de statut et suppression.
GET /api/reparations , GET /api/reparations/{id}	Réparations	Historique des interventions.
POST /api/reparations , PUT /api/reparations/{id} , DELETE /api/reparations/{id}	Réparations	Gestion des interventions ; une réparation résout la panne liée.
GET /api/relevés , GET /api/relevés/{id}	Relevés	Consultation des relevés et filtrage des relevés non facturés.
POST /api/relevés , PUT /api/relevés/{id}	Relevés	Saisie et mise à jour des lectures de compteurs.
GET /api/factures , GET /api/factures/{id}	Facturation	Liste et détail des factures.
POST /api/factures	Facturation	Génère une facture à partir d'un relevé non facturé.
GET /api/factures/{facture}/preview	Facturation	Affiche le PDF de facture dans le navigateur.
GET /api/factures/{facture}/pdf	Facturation	Télécharge le PDF de facture.
POST /api/factures/{facture}/paiements	Paiements	Ajoute un paiement sur une facture.
GET /api/paiements	Paiements	Liste les règlements enregistrés.
GET /api/tariff-settings , PUT /api/tariff-settings	Configuration	Lit et met à jour la grille tarifaire JSON.
GET /api/users , POST /api/users , PUT /api/users/{id}	Utilisateurs	Consultation et gestion des comptes.

Endpoint principal	Catégorie	Utilité
GET /api/users/operators et GET /api/plombiers	Utilisateurs	Retourne la liste des opérateurs / plombiers.
GET /api/reports/pannes/pdf	Rapports	Export PDF mensuel des pannes.
GET /api/reports/invoices/pdf	Rapports	Export PDF mensuel des factures.
GET /api/reports/clients/excel	Rapports	Export Excel clients + compteurs.
GET /api/reports/payments/excel	Rapports	Export Excel des paiements.

Le code distingue bien les droits de lecture, de création, de mise à jour et de suppression selon les rôles. Par exemple, `viewer` reste en lecture seule, tandis qu'un `operator` ne peut pas accéder aux factures ni aux paiements.

9. Base de données

La base relationnelle est structurée pour couvrir le cycle de vie complet du métier : référentiel, incidents, interventions, relève, facturation, encaissement et notifications.

Table / modèle	Rôle	Relations clés
<code>users</code>	Comptes applicatifs avec rôle et identifiant de connexion.	1 utilisateur peut être opérateur assigné à plusieurs pannes et réparations.
<code>personal_access_tokens</code>	Tokens Sanctum pour l'API.	Liés aux utilisateurs via relation polymorphe.
<code>secteurs</code>	Zones géographiques, emplacement, tournée, latitude, longitude.	1 secteur possède plusieurs compteurs.
<code>clients</code>	Abonnés et leurs informations administratives.	1 client possède plusieurs compteurs et plusieurs factures.
<code>compteurs</code>	Compteurs physiques rattachés à un client et à un secteur.	1 compteur a plusieurs pannes, relevés et factures.

Table / modèle	Rôle	Relations clés
pannes	Incidents détectés sur les compteurs.	1 panne appartient à 1 compteur et peut être assignée à 1 opérateur.
reparations	Interventions de réparation.	1 réparation appartient à 1 panne et à 1 opérateur/plombier.
relevés	Lectures des index de compteurs.	1 relevé appartient à 1 compteur et peut mener à 1 facture.
factures	Factures détaillées (montants, lignes, snapshot de calcul, échéance, statut).	1 facture appartient à 1 client, 1 compteur, 1 relevé et possède plusieurs paiements.
paiements	Règlements des factures.	1 paiement appartient à 1 facture et à un créateur utilisateur.
tariff_settings	Configuration JSON des tranches, taxes et coefficients.	Utilisée par <code>BillingService</code> au moment de la génération d'une facture.
notifications	Notifications Laravel en base de données (uuid, data JSON, read_at).	Visibilité globale pour admins, personnelle pour les autres rôles.

Relations principales

User 1----	n Panne (assigned_to)
User 1----	n Reparation (id_plombier)
User 1----	n Releve (created_by)
Client 1----	n Compteur
Client 1----	n Facture
Secteur 1----	n Compteur
Secteur 1----	n Panne (via Compteur)
Compteur 1----	n Panne
Compteur 1----	n Releve
Compteur 1----	n Facture
Panne 1----	n Reparation
Releve 1----	1 Facture

Contraintes utiles à retenir

- Suppression logique (`soft deletes`) sur les clients, compteurs, secteurs, pannes et réparations.
- Unicité de `police` , `cadran` , `reference` de facture et `identifiant` utilisateur.
- Contrôle sur les index de relevé : le nouvel index doit être supérieur ou égal à l'ancien.
- Une panne ne peut pas avoir plusieurs réparations actives concurrentes.
- Les statuts de facture (`impayee` , `partielle` , `payee`) sont recalculés après chaque paiement.

10. Authentification et rôles

Connexion

- L'utilisateur se connecte avec `identifiant` et `password` .
- Le backend crée un token Sanctum et le renvoie au frontend.
- Le token est stocké dans le navigateur puis ajouté automatiquement à chaque requête API.
- En cas de `401` , le frontend vide la session locale et force la reconnexion.

Rôles observés dans le code

Rôle	Accès principal
<code>super_admin</code>	Accès complet au backend ; le middleware lui laisse passer toutes les routes.
<code>admin</code>	Accès étendu : utilisateurs, clients, secteurs, compteurs, pannes, réparations, relevés, factures, paiements, tarifs et rapports.
<code>manager</code>	Pilotage opérationnel et commercial : dashboards, pannes, réparations, relevés, factures, paiements, tarifs et rapports.
<code>operator</code>	Accès limité au travail terrain assigné : pannes assignées, réparations autorisées, relevés dans le périmètre lié aux tâches.
<code>viewer</code>	Lecture seule sur les données métier, les factures, les paiements et les rapports.
<code>developer</code>	Rôle technique présent dans le code mais explicitement bloqué en production.

Remarques importantes

- Le `super_admin` est reconnu côté backend ; côté frontend, il est redirigé vers le dashboard admin.
- Le rôle `developer` existe pour des besoins techniques mais le code le bloque en production, aussi bien dans l'authentification que dans le middleware.
- Le contrôle d'accès est doublé : le frontend masque les écrans/actions non autorisés, et le backend refuse réellement l'accès aux routes interdites.

11. Workflow de l'application

1. Un utilisateur ouvre la page de connexion et saisit son `identifiant` ainsi que son mot de passe.
2. Le backend valide le compte et renvoie un token Sanctum, le rôle et le profil.
3. Le frontend redirige automatiquement l'utilisateur vers le dashboard correspondant à son rôle.
4. Les administrateurs et managers préparent le référentiel : secteurs, utilisateurs, clients, compteurs et paramètres tarifaires.
5. Une panne est déclarée puis éventuellement affectée à un opérateur.
6. L'opérateur consulte ses tâches, traite la panne et peut enregistrer une réparation.
7. Lors des tournées, un relevé est saisi ; la consommation est calculée automatiquement et l'index du compteur est mis à jour.
8. Un administrateur ou manager génère ensuite une facture à partir d'un relevé non encore facturé.
9. Les paiements sont enregistrés au fil des encaissements ; le statut de la facture passe de `impayee` à `partielle` ou `payee`.
10. Les responsables et utilisateurs autorisés consultent enfin les dashboards, la carte des secteurs, les notifications et les rapports exportables.

12. Exemples de code importants

Exemple 1 : authentification et création du token

```
$validated = $request->validate([
    'identifiant' => ['required', 'string'],
    'password' => ['required', 'string'],
    'device_name' => ['nullable', 'string', 'max:255'],
]);

$user = User::where('identifiant', $validated['identifiant'])->first();

if (! $user || ! Hash::check($validated['password'], $user->password)) {
    throw ValidationException::withMessages([
        'identifiant' => ['The provided credentials are incorrect.'],
    ]);
}

$token = $user->createToken($validated['device_name'] ?? 'api-token')->plainTextToken;
```

Cet extrait d' `AuthController` montre la séquence principale : validation des champs, recherche de l'utilisateur, vérification du mot de passe, puis création du token d'API.

Exemple 2 : calcul automatique de la consommation

```
protected static function booted(): void
{
    static::saving(function (Releve $releve): void {
        $releve->consommation = max(
            0,
            (float) $releve->nouvel_index - (float) $releve->ancien_index
        );
    });
}
```

Le modèle `Releve` calcule automatiquement la consommation au moment de l'enregistrement. Cela évite de dépendre du frontend pour une règle métier aussi importante.

Exemple 3 : filtrage métier pour les opérateurs

```
public static function scopeFactures(Builder $query, User $user): Builder
{
```

```

return $query->where(function (Builder $query) use ($user): void {
    $query->whereHas('releve', fn (Builder $releve) => $releve->where('created_by', $user->id))
        ->orWhereHas('compteur.pannes', fn (Builder $panne) => $panne->where('assigned_to', $user-
>id));
});
});
}

```

Dans `OperatorAccess`, les opérateurs ne voient pas toutes les factures : ils sont limités aux éléments liés à leurs propres relevés ou à leurs pannes assignées.

Exemple 4 : structure de calcul d'une facture

```

$waterLines = $this->tariffs->splitConsumption($consumption, data_get($config, 'water.tranches', []),
'water', $waterTva);
$sanitationLines = $this->tariffs->splitConsumption($consumption, data_get($config,
'sanitation.tranches', []), 'sanitation', $sanitationTva);
|
$lineItems = array_values(array_filter(
    [...$waterLines, ...$sanitationLines, ...$fixedLines, ...$taxLines],
    fn (array $line): bool => (float) $line['montant_ht'] > 0
));
|
return [
    'montant_ht' => $montantHt,
    'taxes' => $taxes,
    'tva' => $tva,
    'total_ttc' => $total,
    'line_items' => $lineItems,
    'detail_snapshot' => [
        'agence' => $config['agence'] ?? 'SRM Taza/Region',
        'consommation_m3' => $consumption,
    ],
];

```

Le service `BillingService` assemble des lignes de facturation détaillées par section (`water`, `sanitation`, `taxes`) et produit un `detail_snapshot` enregistré dans la facture pour garder une trace fidèle du calcul.

13. Conclusion

D'après le code source, `RADEETA-Management-System` est une application de gestion métier complète, pensée pour relier les opérations terrain, la gestion des abonnés et la facturation dans un seul système. L'architecture séparée React / Laravel est cohérente, la sécurité par rôles est appliquée à plusieurs niveaux et la logique métier essentielle est bien centralisée dans le backend.

Le projet se distingue aussi par des fonctionnalités de présentation utiles dans un contexte professionnel : cartes GIS, dashboards par rôle, exports PDF/Excel, notifications, seeders de démonstration et premiers tests d'API. En résumé, il s'agit d'une base solide pour un outil de pilotage opérationnel et commercial d'un service de distribution d'eau.